

Урок 3. Корутины. Suspend функции

27 ноября 2019

В этом уроке подробно разберем как создать suspend функции. Также рассмотрим, можно ли блокировать поток, как корутина может потеряться и зачем нужно слово suspend.

В прошлом уроке мы подробно рассмотрели взаимодействие Continuation и suspend функции со стороны Continuation. Теперь посмотрим на это взаимодействие со стороны suspend функций.

Как создать suspend функцию

Сейчас мы будем рассматривать, как создать suspend функцию из асинхронного кода. Если же у вас есть какой-то синхронный метод и его надо сделать suspend, то там просто используется билдер withContext. Об этом мы поговорим, когда начнем тему билдеров.

Давайте создадим suspend функцию download, которую мы использовали ранее в примерах.

```
1 suspend fun download(url: String): File {  
2  
3 }
```

Ключевое слово здесь - suspend. Это даст Котлину понять, что это suspend функция и она будет приостанавливать корутину. В конце урока я дам более подробное объяснение, зачем нужно использовать слово suspend. Но перед этим нам нужно понять, как такая функция работает.

Предположим, у нас есть некий NetworkService, который асинхронно умеет загружать файлы. И мы хотим обернуть его в suspend функцию download:

```
1 suspend fun download(url: String): File {  
2     networkService.download(url, object: NetworkService.Callback {  
3         override fun onSuccess(result: File) {  
4  
5         }  
6     })  
7 }
```

В колбэк onSuccess придет загруженный файл.

Как вы помните из прошлого урока, suspend функция должна результаты своей работы передать в Continuation.invokeSuspend. Для этого используется метод Continuation.resume

```
1 suspend fun download(url: String): File {  
2     networkService.download(url, object: NetworkService.Callback {  
3         override fun onSuccess(result: File) {  
4             continuation.resume(result)  
5         }  
6     })  
7 }
```

Метод resume содержит определенную логику, часть которой - это вызов метода invokeSuspend с передачей туда результата.

Осталось где-то взять continuation. Для этого мы используем функцию suspendCoroutine.

```
1 suspend fun download(url: String): File {  
2     return suspendCoroutine { continuation ->  
3         networkService.download(url, object: NetworkService.Callback {  
4             override fun onSuccess(result: File) {  
5                 continuation.resume(result)  
6             }  
7         })  
8     }  
9 }
```

Весь наш код уходит в блок suspendCoroutine. Этот блок дает нам доступ к continuation, в который мы передаем результат работы networkService.download.

Получается, что suspend функция выполнила асинхронную работу и результат передала в continuation.resume, а тот уже передаст его в continuation.invokeSuspend.

Возврат ошибки

Кроме успешного результата, continuation может принять данные об ошибке.

Пусть у NetworkService.Callback есть метод onFailure. Он будет вызван, если при загрузке произошла ошибка. В нем мы вызовем метод continuation.resumeWithException и передадим туда исключение.

```

1 suspend fun download(url: String): File {
2     return suspendCoroutine { continuation ->
3         networkService.download(url, object: NetworkService.Callback {
4             override fun onSuccess(result: File) {
5                 continuation.resume(result)
6             }
7
8             override fun onFailure(error: Exception) {
9                 continuation.resumeWithException(error)
10            }
11        })
12    }
13 }

```

В этом случае ошибка не пойдет в метод `invokeSuspend`, а будет обработана корутиной. Т.е. в этом случае `continuation` не продолжит свое выполнение и код, который в корутине находится после `suspend` функции не будет выполнен.

Простая `suspend` функция `download` готова. Ее можно вызывать в корутине, она не будет блокировать поток, но приостановит код.

Можно ли сразу вернуть результат

Я все время упоминаю, что `suspend` функция должна сначала выполнить свою работу, а потом вернуть результат. Но возможен и другой вариант в случае, если результат уже готов на момент вызова. В примере с функцией загрузки файла - это может быть использование кэша. `suspend` функция сначала проверяет кэш. Если файл уже есть в кэше, то функция сразу может вернуть его. Для этого ей надо просто сразу вызвать `continuation.resume`. А если файла в кэше нет, то действуем по старой схеме - стартуем асинхронную работу и по ее завершению вызываем `continuation.resume`.

Давайте посмотрим как в Kotlin добавить в `suspend` функцию такую возможность:

```

1 suspend fun download(url: String): File {
2     return suspendCoroutine { continuation ->
3
4         val file = getFileFromCache(url)
5
6         if (file != null) {
7             continuation.resume(file)
8         } else {
9             networkService.download(url, object : NetworkService.Callback {
10                 override fun onSuccess(result: File) {
11                     continuation.resume(result)
12                 }
13
14                 override fun onFailure(error: Exception) {
15                     continuation.resumeWithException(error)
16                 }
17             })
18         }
19     }
20 }
21 }

```

Мы пытаемся получить файл из кэша. Если он там есть, то сразу передаем его в `continuation`, иначе запускаем асинхронную работу.

Блокирование потока

Когда мы создаем `suspend` функцию, мы должны позаботиться о том, чтобы она выполнялась асинхронно в другом потоке или вернула результат сразу. Если же мы напишем в `suspend` функции код, блокирующий поток, то слово `suspend` нам тут никак не поможет. Такая `suspend` функция просто заблокирует поток, в котором выполняется корутина.

Давайте рассмотрим пример некорректной `suspend` функции. Представим, что используемый в примере выше `NetworkService` теперь работает синхронно. Его метод `download` не уходит в фоновый поток и не просит колбэк, а загружает файл в текущем потоке и возвращает как результат вызова метода. Т.е. метод `networkService.download` блокирует поток, в котором он вызван. Если мы используем его напрямую в `suspend` функции, то получится неправильная `suspend` функция:

```

1 // wrong suspend function, don't do that!
2 suspend fun download(url: String): File {
3     return suspendCoroutine { continuation ->
4         val file = networkService.download(url)
5         continuation.resume(file)
6     }
7 }

```

Важно понимать, что ни метод `suspendCoroutine`, ни слово `suspend` в описании функции не сделают наш код асинхронным. Метод `networkService.download` будет выполнен в потоке корутины и заблокирует его. Поэтому от нас требуется обеспечить асинхронность кода в `suspend` функции, а в качестве колбэка использовать `continuation`.

О том, как в корутине выполнять синхронный код так, чтобы не блокировать поток корутины, мы поговорим позже, когда будем рассматривать билдеры.

Потерянная корутина

Что будет если в `suspend` функции не вызвать `continuation.resume`? Это очень важный момент! Если мы не вызовем ни один из методов `continuation.resume*`, то корутина просто не продолжит выполняться. Никогда. Потому что она ждет, что `suspend` функция продолжит ее через вызов `Continuation`. Поэтому тут будьте аккуратны, не теряйте вызов `Continuation` - передайте ему результат или ошибку, чтобы корутина могла продолжиться или завершиться.

Зачем нужно слово `suspend`?

Теперь мы знаем достаточно про `suspend` функции, чтобы поговорить о значении слова `suspend`. Само по себе это слово не добавляет к функции никакой магии. Оно не сделает так, чтобы синхронный код вдруг перестал блокировать поток. Оно вообще ничего не делает. Это просто маркер того, что данная функция умеет (и должна) работать с `Continuation`, чтобы приостановить выполнение корутины не блокируя поток.

Давайте взглянем на это с двух точек зрения: создание `suspend` функции и ее использование.

Создание

Чтобы `suspend` функция могла приостановить код не блокируя поток, ей нужен `Continuation`, выполнение которого она возобновит по завершению своей работы. Чтобы получить `Continuation`, используется функция `suspendCoroutine`. Возникает вопрос, почему любая другая функция без слова `suspend` не может этого сделать? Давайте допустим, что мы можем создать обычную (не `suspend`) функцию и вызвать в ней `suspendCoroutine`, который предоставит нам `Continuation`. Если мы вызовем такую функцию в корутине, то все отработает нормально, потому что у корутины есть `Continuation`. Его мы и получим. Но если мы вызовем ее вне корутины, то `suspendCoroutine` не сможет предоставить нам `Continuation`, потому что его просто нет.

Получается, что функция, в которой мы хотим получить доступ к `Continuation`, должна запускаться только в корутине. В обычном коде ее запускать нельзя. И вот именно для реализации этого ограничения и используется слово `suspend`. Мы не сможем запустить `suspend` функцию вне корутины, компилятор выдаст ошибку. Потому что компилятор знает, что `suspend` функции нужен будет `Continuation`, который есть только в корутине. Таким образом, когда мы создаем функцию и добавляем к ней слово `suspend` - мы можем быть уверены, что она будет запущена в корутине. А значит мы сможем добраться до `Continuation`.

Использование

Если мы в корутине собираемся использовать долго работающую функцию, которая помечена как `suspend`, то мы точно знаем, что эта функция приостановит код и не заблокирует при этом поток, в котором выполняется корутина. Если конечно, она написана корректно)

А обычная функция, которая не является `suspend`, не сможет такого сделать. Она либо заблокирует поток корутины, либо попросит дать ей колбэк.

Что происходит внутри `suspendCoroutine`?

Об этом я подробно расскажу в пятом уроке.